perplexity

Can you create a generic guardrails package in python for RAG applications which will includes all the guardrails rules in it.If answer is yes then provide me a plan in short(don't make it lengthy and not include code also for now).

Yes — this is feasible as a **generic, pluggable Python guardrails package** for RAG applications. It should operate independently of the LLM and cover the full pipeline: query intake, retrieval context, generation, and final response. OWASP recommends input screening, retrieval/context controls, access-aware retrieval, and output validation/redaction for RAG systems. [1] [2]

Proposed package

Name: `rag-guardrails`

Core idea: A configurable pipeline of guardrail rules. Each rule receives a RAG request/context and returns one of: `allow`, `redact`, `warn`, `block`, or `rewrite`.

High-level plan

1. **Define core abstractions**
   - `GuardrailRule` interface with `check()` and optional `transform()`.
   - `GuardrailResult` containing decision, reason, severity, modified text, and metadata.
   - `GuardrailPipeline` supporting ordered input, retrieval, and output stages.
   - `GuardrailConfig` loaded from YAML/JSON/environment variables.
2. **Input guardrails**
   - Query validation: empty, excessive length, malformed input.
   - Prompt-injection and jailbreak detection using regex, heuristics, and optional classifier/LLM judge.
   - Toxicity, hate, self-harm, violence, and unsafe-topic screening.
   - PII and secret detection/redaction in user queries.
   - Rate limiting, abuse-pattern detection, and language/topic allowlists.
   - Query normalization and intent classification.
3. **Retrieval guardrails**
   - Relevance threshold: refuse or degrade if retrieved chunks score too low.

- Minimum/maximum context checks and duplicate-chunk removal.
- Source allowlist/denylist and document freshness checks.
- Access-control enforcement using chunk metadata.
- Context sanitization: strip instructions, delimit untrusted content, and redact sensitive fields.
- Provenance capture: document ID, chunk ID, source URL, timestamp, and retrieval score.

4. **Generation guardrails**
   - Prompt construction controls: system/user/retrieved-content separation and instruction hierarchy.
   - Token/context-window budget enforcement.
   - Tool-call and structured-output schema validation where applicable.
   - Model policy checks: forbidden topics, unsupported claims, and response-length limits.
5. **Output guardrails**
   - Groundedness/faithfulness check: verify claims against retrieved chunks.
   - Citation enforcement and citation validity checks.
   - Hallucination detection using lexical, NLI, embedding, or LLM-judge methods.
   - PII, credential, API-key, and regulated-data redaction.
   - Toxicity, bias, profanity, and unsafe-advice filtering.
   - System-prompt leakage and instruction-leak detection.
   - JSON/schema validation for structured responses.
   - Refusal handling when evidence is insufficient.
6. **Policy and configuration**
   - Rule registry with enable/disable flags.
   - Per-rule thresholds, actions, severity, and custom parameters.
   - Environment-specific policies: development, staging, production.
   - Domain-specific custom rules without changing the package core.
7. **Observability**
   - Structured logs for every guardrail decision.
   - Metrics: block rate, redaction count, grounding score, latency, and rule-level failures.
   - Optional tracing integration with OpenTelemetry.
   - Audit trail linking query, retrieved chunks, model output, and final response.
8. **Testing and packaging**
   - Unit tests for every rule.

    - Adversarial test suite for prompt injection, PII leakage, hallucination, and unsafe content.
    - Integration tests with mock retriever and mock LLM.
    - Publish as an installable Python package with optional extras for heavy dependencies.

Suggested module layout

- `core/` — pipeline, rule interface, results, config.
- `input/` — query and prompt-injection rules.
- `retrieval/` — relevance, access, provenance, and context rules.
- `generation/` — prompt and schema rules.
- `output/` — grounding, PII, toxicity, leakage, and citation rules.
- `redaction/` — PII/secret detection and masking utilities.
- `policies/` — YAML/JSON policy presets.
- `observability/` — logging, metrics, tracing.
- `tests/` — unit, adversarial, and integration tests.

Design principles

- **Fail safe:** block or refuse when confidence or evidence is insufficient.
- **Pluggable:** every rule can be replaced or extended.
- **Stage-aware:** different rules run before retrieval, before generation, and after generation.
- **Non-blocking by default for warnings:** only hard-block on configured critical violations.
- **LLM-agnostic:** works with OpenAI, Anthropic, open-source models, LangChain, LlamaIndex, or custom RAG pipelines.

This aligns with current RAG security guidance: validate inputs, protect retrieved context, enforce access controls, and validate/redact outputs before they reach users or downstream systems. [2] [3] [4]



1. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
2. https://cheatsheetseries.owasp.org/cheatsheets/RAG_Security_Cheat_Sheet.html
3. https://owasp.github.io/DevSecOpsGuideline/3-Governance/3-4-AI-Governance-and-Risk/
4. https://genai.owasp.org/llmrisk/llm072025-system-prompt-leakage/
5. https://qaskills.sh/blog/nemo-guardrails-tutorial-2026
6. https://github.com/Jott2121/rag-guard/blob/main/README.md
7. https://github.com/anmolg1997/Enterprise-RAG-System/blob/main/wiki/Guardrails.md
8. https://github.com/ousmaneb/rag-guardrails
9. https://app.ailog.fr/en/blog/guides/guardrails-rag

10. https://github.com/Jott2121/rag-guard
11. https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html
12. https://docs.aws.amazon.com/de_de/prescriptive-guidance/latest/agentic-ai-security/owasp-top-ten.md
13. https://www.wiz.io/academy/ai-security/owasp-llm-top-10
14. https://genai.owasp.org/wp-content/uploads/2024/05/LLM_AI_Security_and_Governance_Checklist-v1.1.pdf
15. https://www.meilisearch.com/blog/rag-guardrails